Protecting and Tokenising the ALU Logo on Ethereum

Assignment 1 – Cryptocurrency and Digital Assets
BlockChain and Applications

Student: Aman Abraha Kasa

Institution: African Leadership University (ALU)

Term: 2026 September Term

Date: November 2026

GitHub repository:

https://github.com/Aman-Kasa/blockchain_and_applications/tree/main/Assignment-1-cryptocurrency_and_digital_assets

Demo video: https://youtu.be/MWcdP7-wWuk

Contents

1. Abstract
2. Introduction
3. Question 1 – Exploring a Cryptocurrency Exchange
4. Blockchain Setup
5. Smart Contract Explanation
6. Tokenisation
7. Test Results
8. Testnet Deployment (Sepolia)
9. Conclusion
10. GitHub and Video Links

1. Abstract

This project registers the official ALU logo as a non-fungible token (NFT) on the Ethereum blockchain and protects its integrity with a permanently stored SHA-256 fingerprint, so anyone can check whether a logo file is the authentic original or a modified fake. It also tokenises ownership of the logo into 1,000,000 fixed ERC-20 shares (`ALUT`) that ALU can distribute to its stakeholders. Both contracts were built and tested with Hardhat, then deployed and verified on the Sepolia testnet.

2. Introduction

A logo is one of a university's most valuable identity assets, and it is easy to copy, edit, or falsely claim online. ALU needs a way to prove which file is the official logo, who registered it, and when.

A normal database cannot fully provide this. It is centralised, so the owner, a careless administrator, or an attacker who compromises an admin account can secretly edit or delete records, and outsiders simply have to trust that the data is correct. Standard copyright registration gives legal protection, but it relies on slow paperwork, is handled country by country, and does not let anyone instantly check whether a specific image file is genuine.

Registering the logo on a blockchain solves these problems. The record is **immutable** (it cannot be changed after it is written), **public** (anyone can verify it), and **trustless** (no single organisation controls it). Storing the logo's SHA-256 hash means even a one-pixel modification can be detected immediately, and recording the registrant's wallet address and timestamp creates a permanent proof of ownership that nobody can forge.

3. Question 1 – Exploring a Cryptocurrency Exchange (Coinbase)

Step 1 – Creating an account and verifying identity

I signed up on Coinbase with my email and verified my phone number (+250). Coinbase then asked for my citizenship and country of residence, my city and country of birth ("Coinbase is legally required to collect this info"), and finally a passport or government-issued photo ID, along with consent to collect biometric data for a selfie check. It also warned that proof of address might be needed.

Exchanges do this because of **KYC (Know Your Customer)** and **AML (Anti-Money Laundering)** rules. Coinbase is a US company, so it must follow the **Bank Secrecy Act**, enforced by FinCEN, as well as the international **FATF** standards. Linking every account to a real, accountable person makes it much harder to launder money, fund terrorism, avoid sanctions or commit fraud.

One thing stood out to me. Coinbase's accepted-documents table lists **only a passport for Eritrea**, with proof of address "not available", while Ethiopia accepts national IDs, residence permits, bank statements and more. Compliance systems rate countries by risk, and that rating decides how easy verification is, so access to crypto partly depends on where you come from.

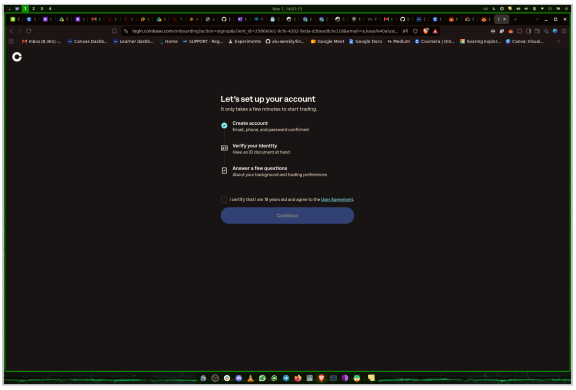


Figure 1 – Coinbase's three-stage onboarding: account, identity verification, questions.

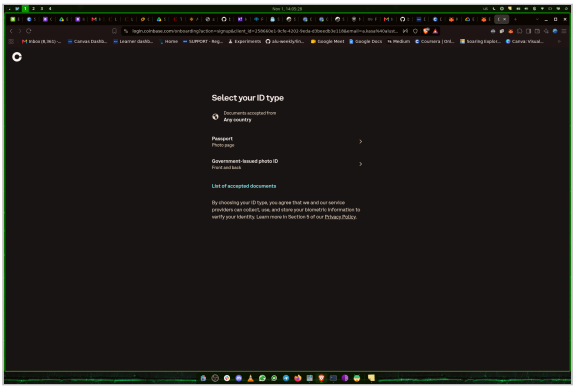


Figure 2 – Accepted ID types and biometric consent.

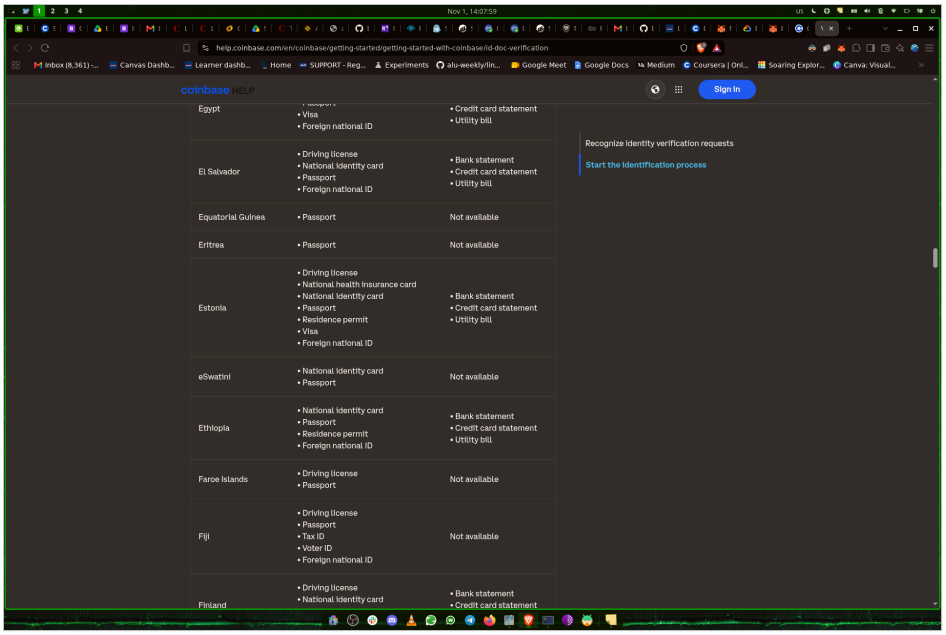


Figure 3 – Accepted documents by country: Eritrea (passport only) compared with Ethiopia.

Step 2 – Three cryptocurrencies

Coinbase showed prices in **RWF** because it detected my location, and it lists over 22,000 assets. I looked at three:

- **Bitcoin (BTC), about RWF 114.8 million:** the first cryptocurrency (2009), a peer-to-peer form of money secured by **Proof of Work** mining. Its supply is capped at **21 million coins**, which is why many people treat it as "digital gold".
- **Ethereum (ETH), about RWF 3.34 million:** a programmable blockchain that runs **smart contracts** on the Ethereum Virtual Machine, with ETH used to pay gas fees. Unlike Bitcoin, it is a platform for applications, and it switched to **Proof of Stake** in 2022. I deployed my own contracts on its Sepolia testnet.
- **Tether (USDT), about RWF 1,474 (≈ $1):** a **stablecoin** pegged to the US dollar and backed by reserves held by the company Tether. Its price changed only **−0.12% in a year**, compared with about −30% for BTC and −43% for ETH, so people use it to hold value or move money without the volatility.

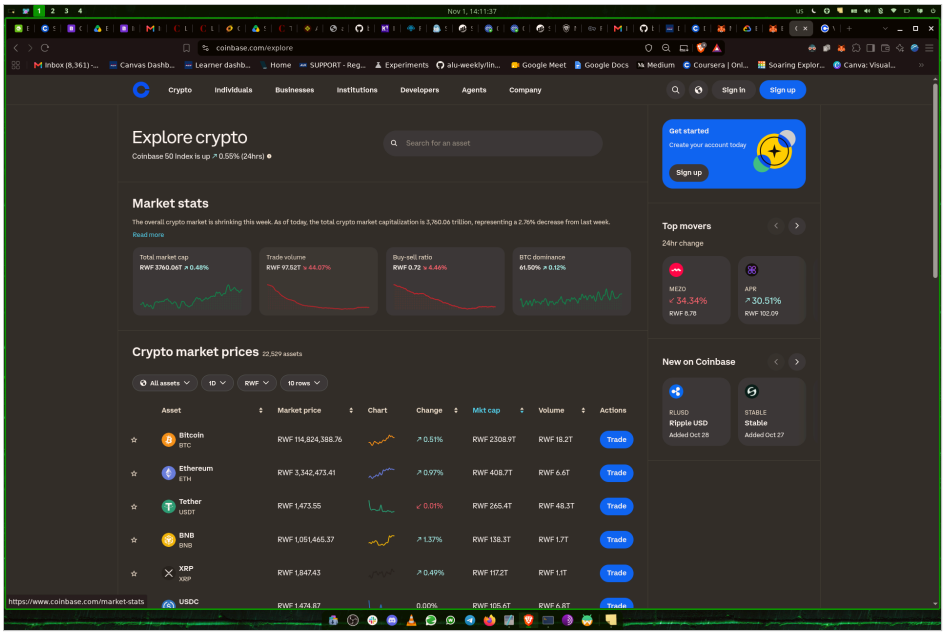


Figure 4 – Coinbase "Explore crypto" page with prices in RWF.

Step 3 – Adding a payment method

You can only add a payment method after verifying your account, and the card or bank account must be **in the same name** as your Coinbase account; mismatches are reviewed manually for 1–3 business days. A debit card needs the card number, expiry date, CVV and a **3D Secure** confirmation code from your bank. A bank transfer needs your account number or IBAN.

Coinbase's tables show that **cards are instant**, while **bank transfers (such as SEPA) take 1–3 business days**. Cards usually carry higher fees, so the card is faster and the bank transfer is cheaper.

For Rwanda, I noticed that Coinbase only publishes payment tables for the **EU, UK and UAE**. I found no African option such as mobile money, and Coinbase warns that banks may convert currency at their own exchange rate, which is a hidden extra cost for someone paying in RWF.

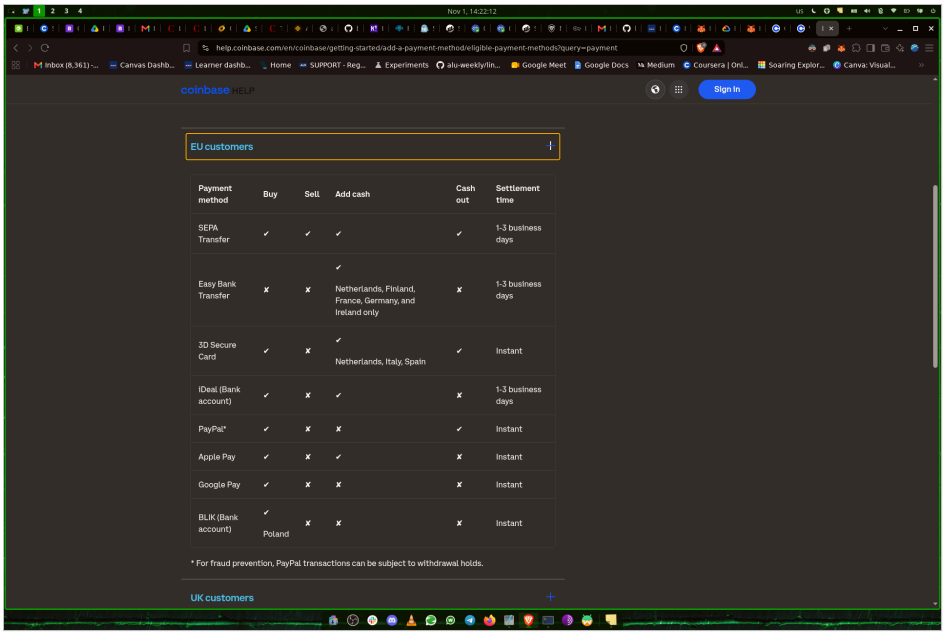


Figure 5 – Eligible payment methods for EU customers: cards are instant, bank transfers take 1–3 days.

Step 4 – Simulating a $100 purchase

A **market order** buys immediately at the current price. A **limit order** only executes if the price reaches a level you set, so it gives you control but might never be filled.

I would choose a **market order**. On Coinbase's simple interface, a limit order adds an **extra 1% fee** on top of the normal Coinbase fee (up to 1.875%), so for a small $100 purchase the market order is both faster and cheaper.

With Bitcoin at about **$77,819**, $100 minus a 1.875% fee leaves $98.13, which buys roughly **0.00126 BTC**. The final amount is slightly lower because of the spread, which Coinbase shows on the preview screen before you confirm.

Step 5 – Transaction history

The Transactions tab lists open orders and completed orders grouped by month, and you can filter by type, status, asset and date. Each entry shows a **reference code, payment method and fee**, and you can download monthly statements.

A **transaction hash** is different. It is the unique ID of a transaction recorded **on a blockchain**, created by hashing the transaction's data. Trades inside Coinbase only get an internal reference code; a hash appears when crypto actually moves on-chain, for example during a withdrawal. Anyone can paste a hash into a block explorer to verify that the transaction happened. When I registered the ALU logo on Sepolia, my transaction hash (`0xb6a7e459…c9a34415`) let me see the block number, sender, gas used and events on Etherscan.

Step 6 – Security features

2-step verification (2FA) is required on every Coinbase account. The methods range from hardware **security keys** and **passkeys** to **authenticator apps** and **SMS**. I would use a security key plus an authenticator app and avoid SMS, because SIM-swap attacks

can intercept text-message codes, while a hardware key keeps its secret inside the device where it cannot be phished remotely.

Withdrawal holds: for fraud prevention, Coinbase can delay some withdrawals, for example PayPal transactions. If someone breaks into an account, they cannot drain the funds instantly, which gives the real owner time to notice and react.

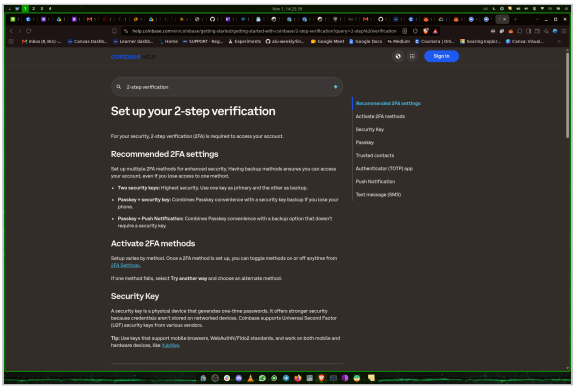


Figure 6 – Coinbase 2-step verification methods and recommended settings.

4. Blockchain Setup

I used **Hardhat** to run a local Ethereum blockchain on my own computer (Pop!_OS Linux, Node.js v20). A local blockchain simulates the Ethereum network on a single machine, so I could compile, test and deploy my contracts for free and instantly, without spending real ETH or waiting for public blocks. Mistakes cost nothing and the chain can be reset at any time, which makes it the right place to develop before going to a public network.

```
npm install                 # Hardhat 2.29.1, Hardhat Toolbox, OpenZeppelin 5
npm run hash                # SHA-256 of alu-logo.png
npx hardhat compile         # Solidity 0.8.28, evmVersion: cancun
npx hardhat test            # 8 automated tests
npx hardhat node            # local chain at http://127.0.0.1:8545 with 20 funded accounts
npx hardhat run scripts/deploy.js --network localhost
```

What each part of the setup does:

Part	What it does
Node.js v20 + npm	Runs JavaScript on my machine and installs the project's packages.
Hardhat 2.29.1	Development framework: compiles Solidity, runs tests, runs a local blockchain and deploys contracts.
OpenZeppelin Contracts 5	Audited, standard ERC-721, ERC-20 and Ownable code that my contracts extend.
`hardhat.config.js`	Sets the compiler (0.8.28, EVM `cancun`, optimizer) and the networks (localhost, Sepolia).
`contracts/`, `scripts/`, `test/`	The Solidity contracts, the deployment and hashing scripts, and the 8 automated tests.
`npx hardhat node`	Starts a local Ethereum node at `127.0.0.1:8545` that mines a block for every transaction.

```
Started HTTP and WebSocket JSON-RPC server at http://127.0.0.1:8545/

Account #0: 0xf39Fd6e51aad88F6F4ce6aB8827279cffFb92266 (10000 ETH)
Account #1: 0x70997970C51812dc3A010C7d01b50e0d17dc79C8 (10000 ETH)
Account #2: 0x3C44CdDdB6a900fa2b585dd299e03d12FA4293BC (10000 ETH)
... (20 accounts in total)
```



Figure 7a – `npx hardhat node` running locally with 20 funded test accounts.

When `npx hardhat node` starts, it creates **20 test accounts**, each funded with 10,000 fake ETH. Each account has a **wallet address**, a public identifier derived from the account's public key (the last 20 bytes of its Keccak-256 hash). An address represents an account on the blockchain: it can receive and send ETH and tokens, deploy contracts, and own NFTs. Only the holder of the matching private key can sign transactions from it.

The deploy script used Account #0 (`0xf39F…2266`) and produced this output on the local chain:

```
ALU logo SHA-256: 0xa833391cae5c6ba476d77d339c3f7100d96dd9e7fea3faed4b6efaafba2452c5
ALUAssetRegistry deployed to: 0x5FbDB2315678afecb367f032d93F642f64180aa3
ALU logo registered as token ID: 1
Integrity check: true - Logo is authentic.
ALULogoToken deployed to: 0x9fE46736679d2D9a65F0992F2272dE9f3c7fa6e0
Deployer ALUT balance: 1000000
```

The node's log showed that each state-changing transaction created a new block (registry deployment in block 1, logo registration in block 2, token deployment in block 3), while the read-only `verifyLogoIntegrity` call created no block and used no gas.

5. Smart Contract Explanation

A **smart contract** is a program stored on a blockchain that runs exactly as written on a decentralised virtual machine (the EVM), executed and checked by every node. A normal server program runs on a central server controlled by one company, which can change the code, edit the data, or shut it down. Once deployed, a smart contract's rules are public and cannot be secretly altered.

5.1 Generating the ALU logo hash

I generated the SHA-256 hash of the actual logo file in two independent ways, with the project script (`npm run hash`, using Node's `crypto` module) and with the Linux tool `sha256sum`. Both gave the same result:

```
a833391cae5c6ba476d77d339c3f7100d96dd9e7fea3faed4b6efaafba2452c5
```

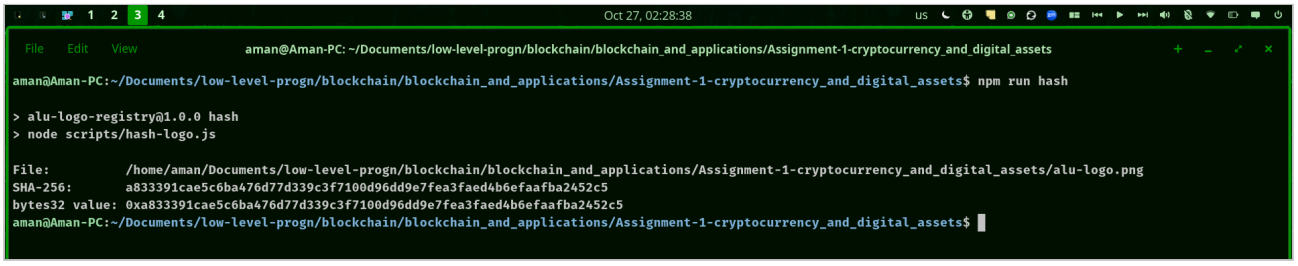


Figure 7 – SHA-256 hash of alu-logo.png generated with the project script.

SHA-256 turns any file into a fixed 256-bit fingerprint. Changing even one pixel changes the input bytes, and because of the **avalanche effect** the output changes completely and unpredictably. A modified logo can never produce the original hash, which is what makes the integrity check reliable.

5.2 ALUAssetRegistry (ERC-721)

An **NFT** is a unique digital token, where each token ID is different and cannot be swapped one-for-one with another. **ERC-721** is the standard for this, which makes it the right choice for registering a logo: the official ALU logo is one specific, unique asset, so it should be represented by exactly one token with its own owner and metadata. The contract extends OpenZeppelin's audited ERC-721 implementation.

Component	Plain-English explanation
`struct AssetMetadata`	A record for each asset: name, file type, SHA-256 content hash (`bytes32`), the wallet that registered it, and the Unix timestamp of registration.
`mapping(uint256 => AssetMetadata)`	Looks up the registration details for any token ID.
`mapping(bytes32 => bool)`	Remembers which hashes are already registered, so the same file can never be registered twice.
`registerAsset(name, fileType, contentHash)`	Rejects an empty or already-used hash, increases the token counter, saves the metadata, marks the hash as used, mints the NFT to the caller with `_safeMint()` , emits `AssetRegistered` , and returns the new token ID.
`verifyLogoIntegrity(tokenId, contentHash)`	Read-only (`view`), so it costs no gas. Compares the supplied hash with the stored one and returns `true, "Logo is authentic."` or `false, "Warning: logo does not match."`
`getAsset(tokenId)`	Read-only. Returns the full `AssetMetadata` record so anyone can look up the registration.
`isRegistered(contentHash)`	Extra helper that checks whether a hash has already been registered.

Security choices: the contract saves all data *before* calling `_safeMint()` (the checks-effects-interactions pattern), because `_safeMint()` can call back into a receiving contract; this prevents re-entrancy from registering the same hash twice. It also rejects zero hashes and reverts when a token ID does not exist, instead of returning empty data.

5.3 Why on-chain data cannot be changed

Each block contains the hash of the previous block, so blocks are cryptographically chained together. Changing an old record would change that block's hash and break every block after it. Thousands of nodes also keep their own copies and only accept blocks that follow the consensus rules, so a single party cannot rewrite history. Once the logo's hash was recorded in block 12,040,640 on Sepolia, it became permanent.

6. Tokenisation

Tokenisation means splitting ownership of an asset into many digital tokens. Instead of one party owning the whole ALU logo, ownership can be shared among many holders, each owning a precise, transferable fraction recorded on the blockchain.

6.1 ALULogoToken (ERC-20)

Component	Plain-English explanation
Name / symbol	"ALU Logo Token" / `ALUT`
Constructor	Mints the full fixed supply of **1,000,000 ALUT** to the `logoOwner` address and makes that address the contract owner. No other mint function exists, so the supply can never grow.
`decimals() = 0`	Each token is one whole, indivisible share, so 100,000 means exactly 100,000 shares.
`distributeShares(recipient, amount)`	Restricted with `onlyOwner`. Requires an amount greater than zero and a valid recipient, transfers shares from the owner, and emits `SharesDistributed`.
`ownershipPercentage(account)`	Read-only. Returns the holder's share of the total supply as a whole-number percentage (500,000 tokens → 50).

6.2 Who at ALU would hold ALUT, and what it means

ALU could distribute ALUT to **administration** (to manage the brand), **faculty** (recognising their contribution to the university's reputation), and **students or alumni** (giving the community a stake in the institution's intellectual property). Holding tokens could, for example, be used to give holders a say in decisions about how the logo is licensed or used.

If a faculty member holds **100,000 out of 1,000,000 ALUT**, they hold **10% of the ownership shares** of the logo. In practice, their stake is recorded transparently on-chain, it can be checked at any time with `ownershipPercentage()`, and it can be transferred. Any legal rights attached to those shares, such as a share of licensing income, would still need to be defined by ALU's own policies, because the token records ownership but does not create legal rights on its own.

6.3 ERC-721 vs ERC-20

	ERC-721 (registry)	ERC-20 (ALUT)
Type	Non-fungible, every token is unique	Fungible, every token is identical
Represents	The one official logo, with its own metadata	Interchangeable ownership shares
Supply	One token per registered asset	1,000,000 fixed shares
Question it answers	*What* is the asset, and who registered it?	*Who* owns how much of it?

I used ERC-721 for registration because the logo is a single unique asset that needs its own identity and fingerprint, and ERC-20 for the shares because ownership portions must be identical, divisible among people, and easy to transfer.

7. Test Results

All eight automated tests pass (Mocha/Chai with Hardhat Network Helpers). The tests hash the real `alu-logo.png` file, so they check the actual logo rather than a placeholder.

#	Test
1	The ALU logo registers successfully and returns a token ID
2	Registering the same hash a second time is rejected
3	`verifyLogoIntegrity()` returns true for the correct hash
4	`verifyLogoIntegrity()` returns false for an incorrect hash
5	`getAsset()` returns the correct name and file type
6	The full 1,000,000 ALUT supply is minted to the logo owner
7	`distributeShares()` transfers tokens (and blocks non-owners and zero amounts)
8	`ownershipPercentage()` returns the correct percentage

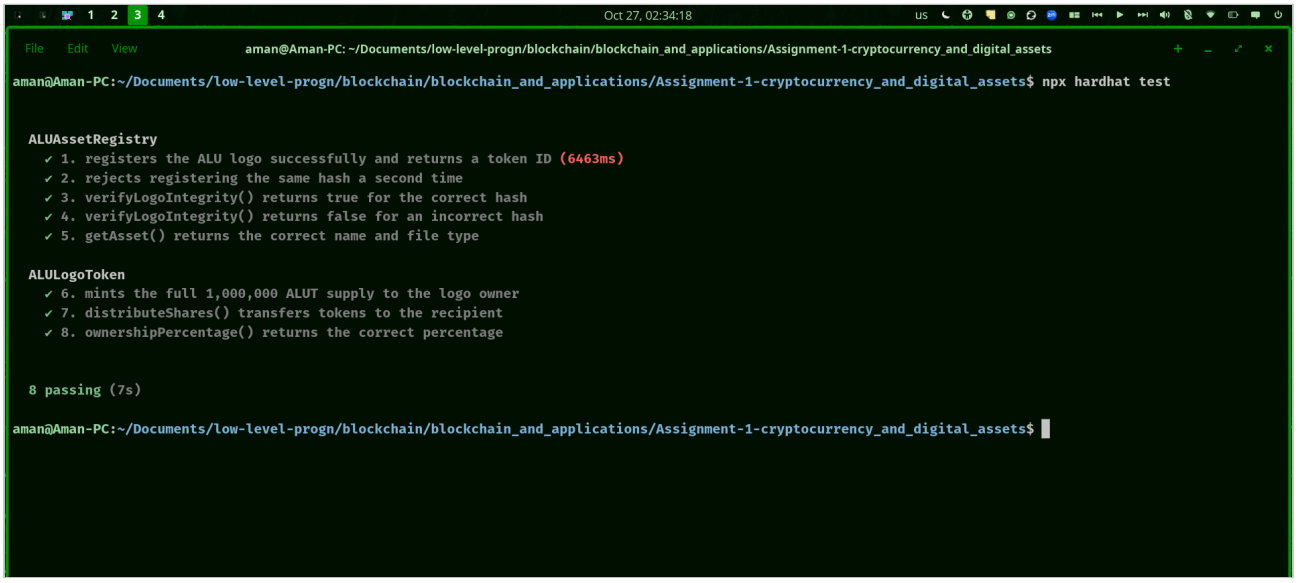


Figure 8 – All 8 tests passing (`npx hardhat test`).

8. Testnet Deployment (Sepolia)

Both contracts were deployed to the Sepolia testnet through Remix IDE with MetaMask, using the same compiler settings as Hardhat (Solidity 0.8.28, EVM `cancun`, optimizer on, 200 runs). Remix also verified both contracts on **Sourcify** with an **Exact Match**, so anyone can confirm that the deployed bytecode is exactly the source code in the repository.

Item	Address / transaction hash
ALUAssetRegistry	`0x6caC78575f42e232017B602F11dfF61456D89E70`
ALULogoToken	`0x835292dDDd678cCf2447321CA354E5A490DB69E2`
Logo registration (`registerAsset` , block 12,040,640)	`0xb6a7e4596594a7b74952430edf4e8c99f928b4f1ee9a0bc04be26bd5c9a34415`
Token deployment (block 12,040,723)	`0xaaad338f9128b1aa84381e955b97a012f59b6828e8b623ec659e9ee8a4f862c5`
`distributeShares` (block 12,040,799)	`0xd438e5b62f9753c7da0d520370ced9f1e53d640e300cdc570489a47744430e05`

Results verified on the live testnet:

Call	Result
`verifyLogoIntegrity(1, 0xa833…52c5)`	`true` , "Logo is authentic."
`verifyLogoIntegrity(1, 0xa833…52c6)` – one character changed	`false` , "Warning: logo does not match."
`getAsset(1)`	ALU Official Logo, image/png, `0xa833…52c5` , `0x605e…690C` , 1793531712 (1 Nov 2026, 13:15:12 CAT)
`balanceOf(owner)` after deployment	1,000,000 ALUT
`ownershipPercentage` after distributing 100,000 ALUT	Owner 90%, stakeholder 10%

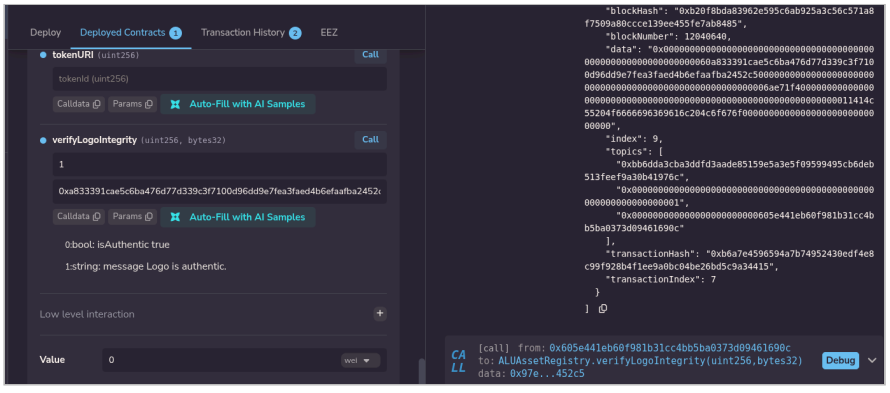


Figure 9 – verifyLogoIntegrity returns true for the real logo hash.

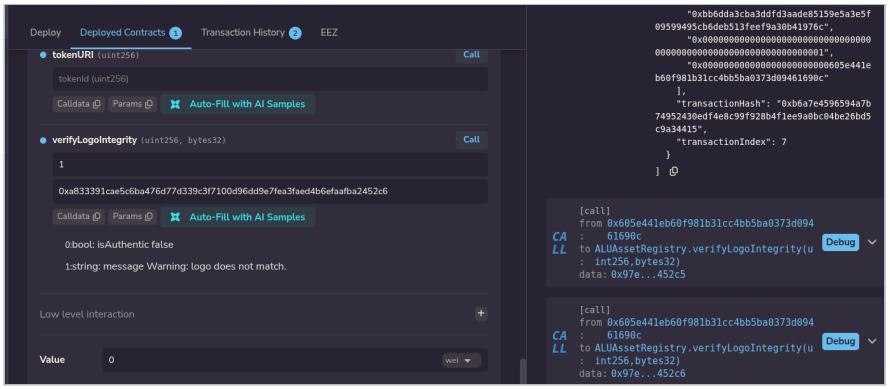


Figure 10 – Changing one character of the hash returns false.

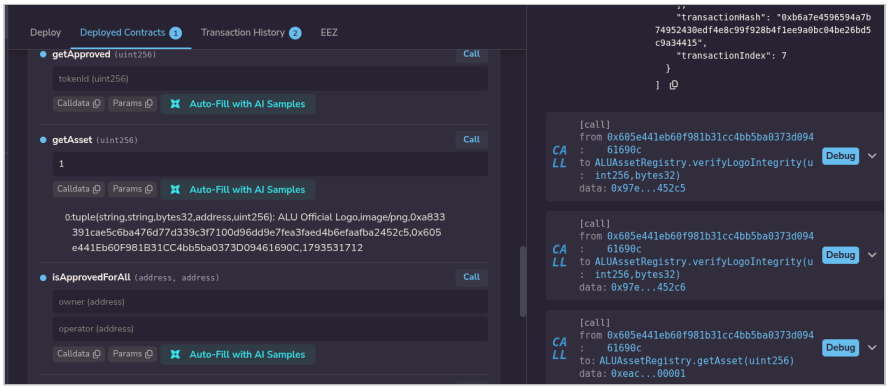


Figure 11 – getAsset(1) returns the full registration record.

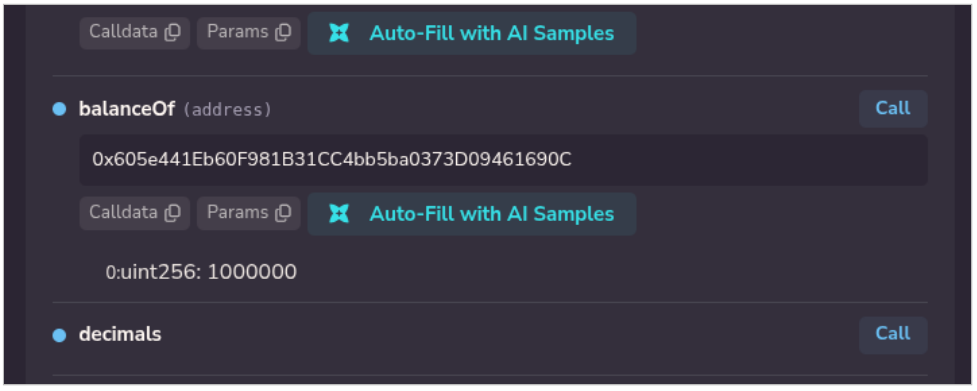


Figure 12 – The logo owner holds the full 1,000,000 ALUT supply.

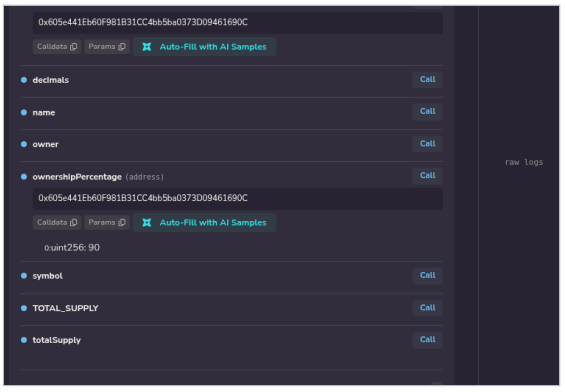


Figure 13 – After distributing 100,000 ALUT, the owner holds 90%.

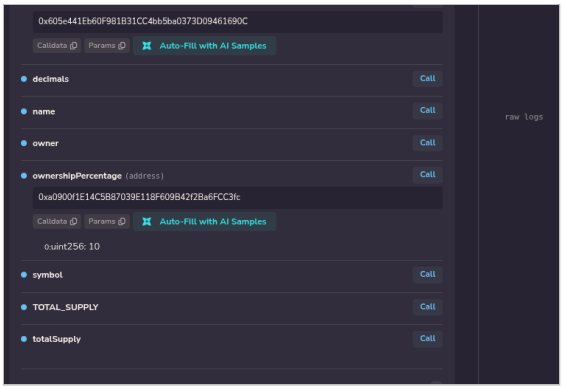


Figure 14 – The stakeholder holds 10%.

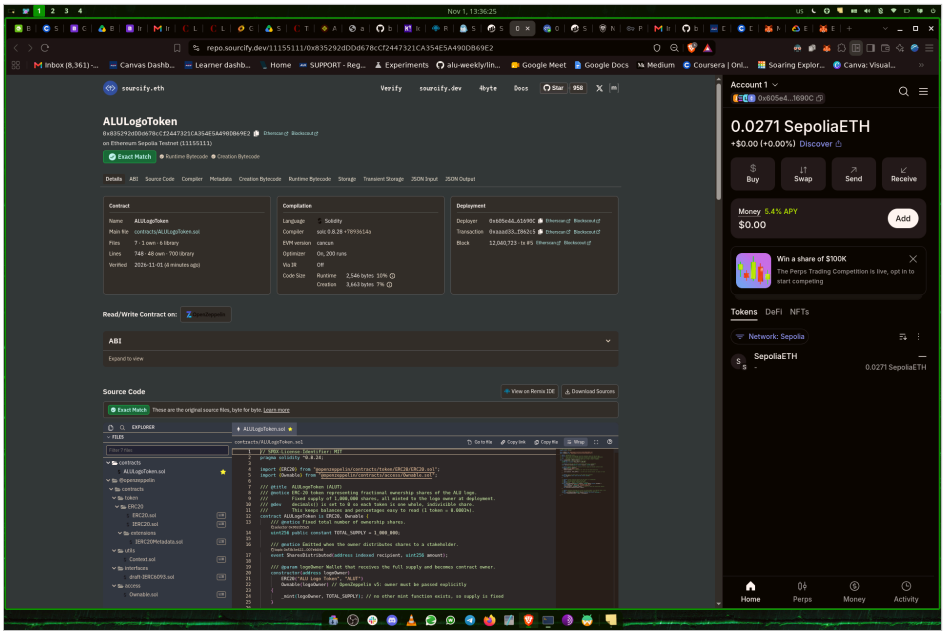


Figure 15 – ALULogoToken verified on Sourcify (Exact Match).

9. Conclusion

This project showed me how blockchain can protect a real digital asset. I learned how to design Solidity contracts with structs and mappings, extend audited OpenZeppelin standards, write automated tests, deploy with Hardhat, and use cryptographic hashing to prove a file's integrity. Seeing the integrity check return `false` after changing a single character of the hash made the avalanche effect very concrete.

The hardest part was debugging my development environment rather than the contracts themselves. My `/home` partition ran out of space during `npm install`; compilation failed with `Function "mcopy" not found` until I upgraded to Solidity 0.8.28 and set `evmVersion: "cancun"` for OpenZeppelin 5; and Remix could not detect MetaMask in Brave until I installed the extension there and disabled Brave's built-in wallet. Solving these taught me to read error messages carefully and to check tool versions.

With more time, I would build a React dApp front end where anyone can upload a logo file, hash it in the browser and verify it against the blockchain; store the logo file itself on **IPFS** and link it through `tokenURI`; and add on-chain governance so ALUT holders could vote on how the logo is licensed.

10. GitHub and Video Links

GitHub repository:

https://github.com/Aman-Kasa/blockchain_and_applications/tree/main/Assignment-1-cryptocurrency_and_digital_assets

Demo video: https://youtu.be/MWcdP7-wWuk